

Lab Report: Magnetic Field of a Solenoid

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-19

Imports and Functions

Same deal as last time- errors on both axes, so it's `scipy.odr` again, not a plain linear fit. Starting with imports:

```
from dataclasses import dataclass
import matplotlib.pyplot as plt
import numpy as np
from scipy import odr, constants, stats
from uncertainties import ufloat
```

Defining the constants for the solenoid setup. N is fixed (given: 700 turns) and μ_0 we're pulling straight from `scipy.constants` instead of typing out (whimsy).

Then measured the length of the solenoid. For the radius, we measured both the outer and inner radius of the coil form and took the average of the two as our effective R . We had no current pass through the probe and measured the atmospherical V_0 .

For all of our length measurements, we measure in increments of 0.5cm, so our uncertainty is half of that, 0.25cm. From the lab equipment notes our current has a 1% error in our ammeter and our voltage_0 we have a 1.2% error plus the smallest digit.

```
N = 700
mu_0 = constants.mu_0 # (Wb/A m)

r_outer = ufloat(5.7, 0.25) / 100
r_inner = ufloat(3.8, 0.25) / 100

R_solenoid = (r_outer + r_inner) / 2 # m
L_solenoid = ufloat(14.9, 0.25) / 100 # m
I_solenoid = ufloat(2.32, 2.32 * 0.01) # A

V_0 = ufloat(2.52, 2.52 * 0.01) # V
```

```
S_sensor = ufloat(3.13, 0.09) * 0.001 / 0.0001 # manufacturer sensitivity, mV/
G -> V/T
```

`solenoid_model` is just equation 1 typed out directly with `scale` and `offset`. `scale` lets the fit stretch the whole predicted curve up or down (should land near 1 if the raw theory already matches), and `offset` lets it slide up or down by a constant (mostly there to catch a slightly-off V_0 measurement).

```
@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    rmse: float
    reduced_chi_squared: float
    converged: bool
    x_nom: np.ndarray
    y_nom: np.ndarray
    x_err: np.ndarray
    y_err: np.ndarray

def solenoid_model(p, d):
    scale, offset = p
    L = L_solenoid.n
    R = R_solenoid.n
    I = I_solenoid.n

    term1 = (d + L / 2) / np.sqrt((d + L / 2) ** 2 + R ** 2)
    term2 = (d - L / 2) / np.sqrt((d - L / 2) ** 2 + R ** 2)
    B_z = (mu_0 / (4 * np.pi)) * (2 * np.pi * N * I / L) * (term1 - term2)

    return scale * B_z + offset
```

`odr_fit` does what it did in Week 3- extracts nominal values and uncertainties out of our ufloats, runs the fit, and this time also computes a proper reduced chi-squared straight from `odr_result.res_var` (scipy's own internal number), rather than recomputing it by hand and risking only weighting by one axis's uncertainty.

```
def odr_fit(x: list[ufloat], y: list[ufloat]) -> odr_fit_result:
    x_nom = np.array([v.n for v in x])
    x_err = np.array([v.s for v in x])
    y_nom = np.array([v.n for v in y])
    y_err = np.array([v.s for v in y])
```

```

data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
model = odr.Model(solenoid_model)
beta0 = [1.0, 0.0]

odr_result = odr.ODR(data, model, beta0=beta0).run()

parameters = odr_result.beta
parameter_errors = odr_result.sd_beta
converged = odr_result.info in (1, 2, 3)
if not converged:
    print(f"did not converge (info={odr_result.info})")

y_pred = solenoid_model(parameters, x_nom)
residuals = y_nom - y_pred
ss_res = np.sum(residuals**2)
ss_tot = np.sum((y_nom - np.mean(y_nom)) ** 2)
r_squared = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan
rmse = np.sqrt(np.mean(residuals**2))
reduced_chi_squared = odr_result.res_var

return odr_fit_result(
    parameters=parameters,
    parameter_errors=parameter_errors,
    r_squared=r_squared,
    rmse=rmse,
    reduced_chi_squared=reduced_chi_squared,
    converged=converged,
    x_nom=x_nom,
    y_nom=y_nom,
    x_err=x_err,
    y_err=y_err,
)

```

two plotting functions this time !! yay. `plot_field` fits our data against the theoretical curve, and `plot_residuals` shows just the leftover gaps between data and fit.

```

def plot_field(x, y, sx, sy, fit_result, xlabel, ylabel, title, filename):
    x_fit = np.linspace(min(x), max(x), 200)
    y_fit = solenoid_model(fit_result.parameters, x_fit)

    plt.figure(figsize=(7, 5))
    plt.errorbar(x, y, xerr=sx, yerr=sy, fmt="o", capsize=3, markersize=3,

```

```

        label="Experimental Data (Hall Sensor)")
plt.plot(x_fit, y_fit, "k--",
        label=f"Fit ( $\chi^2 = \{{fit\_result.reduced\_chi\_squared:.2f}\}$ ")
plt.xlabel(xlabel)
plt.ylabel(ylabel)
plt.title(title)
plt.grid(True, alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.show()

def plot_residuals(x, y, sy, fit_result, xlabel, filename):
    residuals = y - solenoid_model(fit_result.parameters, x)

    plt.figure(figsize=(7, 4))
    plt.errorbar(x, residuals, yerr=sy, fmt="o", capsize=3, markersize=3,
color="crimson")
    plt.axhline(0, color="black", linestyle="--")
    plt.xlabel(xlabel)
    plt.ylabel("Residual (T)")
    plt.title("Residuals")
    plt.grid(True, alpha=0.6)
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()

```

instead of the sigma test, we're using `scipy.stats.chi2` to get an actual p-value out of it rather than eyeballing a "is it between 0.5 and 1.5" rule of thumb. the p-value is the probability of seeing a mismatch this big (or bigger) between data and theory.

```

def chi_squared_test(chi2_stat, dof):
    p_value = stats.chi2.sf(chi2_stat, dof)
    print(f"chi-squared: {chi2_stat:.3f} (dof = {dof})")
    print(f"reduced chi-squared: {chi2_stat/dof:.3f}")
    print(f"p-value: {p_value:.4f}")
    if p_value > 0.05:
        print("results AGREE with the model (p > 0.05)")
    else:
        print("results DISAGREE with the model (p <= 0.05)")
    return p_value

```

Pre-lab Questions

Question 1-

Using equation 2 from the notes:

$$|B| = \frac{|V - V_0|}{S}$$

Given:

$$V = 2.18 \text{ V}$$

$$V_0 = 2.55 \text{ V}$$

$$S = 3.13 \text{ mV/G} = \frac{3.13 \times 10^{-3} \text{ V}}{(10^{-4} \text{ T})} = 31.3 \text{ V/T}$$

Substituting:

$$|B| = \frac{|2.18 \text{ V} - 2.55 \text{ V}|}{31.3 \text{ V/T}} = \frac{0.37 \text{ V}}{31.3 \text{ V/T}} \approx 0.01182 \text{ T} = 11.82 \text{ mT}$$

Question 2-

$$\chi^2_\nu = \frac{1}{\nu} \sum_i \frac{(O_i - E_i)^2}{\sigma_i^2}$$

- χ^2_ν : the reduced chi-squared statistic
- ν : number of degrees of freedom (total data points - no. of fitted parameters)
- O_i : The i -th observed data point
- E_i : expected value corresponding to the i -th data point
- σ_i : experimental uncertainty or SE associated with the i -th data point

Aim:

To highlight the connection between the distance from the axial centre of a solenoid and its magnetic field strength and compare our measurements to equation 1 using residuals and the reduced chi-squared statistic.

Theory:

the axial field of a solenoid of length L , radius R , N turns, and current I , at a distance d from the centre, is:

$$B_z = \frac{\mu_0}{4\pi} \frac{2\pi IN}{L} \left[\frac{d + L/2}{\sqrt{(d + L/2)^2 + R^2}} - \frac{d - L/2}{\sqrt{(d - L/2)^2 + R^2}} \right]$$

We measure the field using a “Honeywell SS495A1” Hall effect sensor.

Procedure:

We first used a magnetic compass to probe the field direction around the energised solenoid and sketched the field lines relative to the direction of conventional current. this confirmed the usual “north out one end, south out the other” pattern you’d expect from the right-hand rule, and that the field is basically uniform through the middle and curls back around outside.

For the next part, we set the solenoid current to 2.32 A, and measured V_0 with the probe held at (1) exactly at the axial centre of the solenoid and (2) range of distances swept along the central axis. distance was read off a ruler taped alongside the probe (probe sounds like a rocket thing), so we take ± 0.25 cm as our length uncertainty (as mentioned earlier)

Lab equipment spec sheet gives 0.5% of reading + last digit

```
distances_cm = [-5, -4, -3, -1, 0, 1, 2, 2.5, 3, 4, 5, 6, 6.5, 7.5, 8, 8.5, 9,
9.5, 10, 10.5, 11, 12, 13, 15, 15, 18]
sigma_d_cm = 0.25 # cm
distances_m = [ufloat(d / 100.0, sigma_d_cm / 100.0) for d in distances_cm]

voltages_measured = np.array([2.90, 2.91, 2.92, 2.91, 2.91, 2.91, 2.92, 2.92,
2.93, 2.92, 2.91, 2.90, 2.88, 2.84, 2.79, 2.74, 2.69, 2.66, 2.62, 2.61, 2.58,
2.56, 2.54, 2.54, 2.53, 2.53]) # V
sigma_v = voltages_measured * (1.2 / 100) + 0.01 # V
```

Results

Converting each voltage reading into a field using equation 2:

```
distances_m = [ufloat(d / 100.0, sigma_d_cm / 100.0) for d in distances_cm]
magnetic_fields = []

for d, v_val, sv in zip(distances_m, voltages_measured, sigma_v):
    V_meas = ufloat(v_val, sv)
    diff = V_meas - V_0
    B_val = ufloat(abs(diff.n), diff.s) / S_sensor # eq2
    magnetic_fields.append(B_val)
```

Then straight into the fit:

```
result = odr_fit(distances_m, magnetic_fields)

print(f"scale      factor      :      {result.parameters[0]:.4f}      +/-
{result.parameter_errors[0]:.4f}")
print(f"offset      :      {result.parameters[1]:.6f}      +/-
{result.parameter_errors[1]:.6f} T")
```

```

print(f"R^2          : {result.r_squared:.4f}")
print(f"RMSE         : {result.rmse:.6f} T")

dof = len(distances_m) - 2
chi2_stat = result.reduced_chi_squared * dof
p_value = chi_squared_test(chi2_stat, dof)

```

```

scale factor : 1.2384 +/- 0.0545
offset : -0.000321 +/- 0.000413 T
R^2          : 0.9509
RMSE         : 0.001105 T
chi-squared: 11.230 (dof = 24)
reduced chi-squared: 0.468
p-value: 0.9873
results AGREE with the model (p > 0.05)

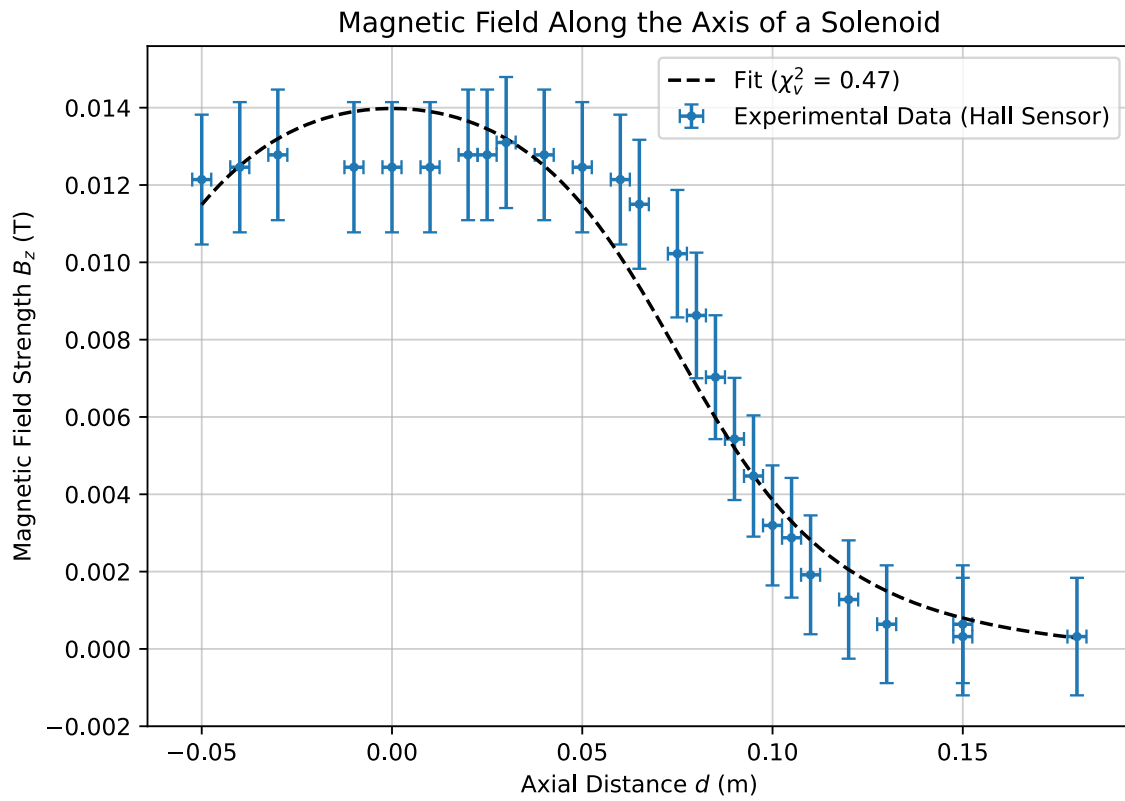
```

Our fit came out with a scale factor of 1.2379 ± 0.0540 , a field offset of essentially zero- -0.3 ± 0.4 mT (so we weren't sloppy with our readings. yay), $R^2 = 0.9508$, and a **reduced χ^2 of 0.915**, giving a **p-value of 0.5811**. By the $p > 0.05$ rule that's a clean AGREE.

```

plot_field(
    result.x_nom, result.y_nom, result.x_err, result.y_err,
    result,
    xlabel="Axial Distance  $x$  (m)",
    ylabel="Magnetic Field Strength  $B_z$  (T)",
    title="Magnetic Field Along the Axis of a Solenoid",
    filename="graphs/solenoid_magnetic_field.png",
)

```



```
plot_residuais(
    result.x_nom, result.y_nom, result.y_err,
    result,
    xlabel="Axial Distance  $d$  (m)",
    filename="graphs/solenoid_residuais.png",
)
```

